ESAT INNOVATION SCHOOL | ESCUELA SUPERIOR DE ARTE Y TECNOLOGÍA

MAM ENGINE

Nombre Asignatura:Graphic Engine Programming (3PMG)
Modulo Asociado:Materia UNIT 46. Advanced Rendering & Visualisation
Profesor:Arnau Rosello
Curso: 2025/2026
Autor/es: Alicia de Cubas, Matias Pedraza, Mark Syniato

Index

1. Engine Features

MAM Engine is a real-time 3D rendering engine developed in C++17. It uses OpenGL 4.6 (core profile) as its primary graphics backend and includes an experimental Vulkan 1.x backend that shares the same high-level rendering API through a common abstraction layer. The engine architecture cleanly separates the rendering pipeline, scene management, asset loading, and application logic into independent, well-defined subsystems.

The engine provides the following core subsystems:

- **Backend-agnostic Render API:** A GraphicsDevice abstract interface defines all GPU operations (create shaders, pipelines, buffers, textures, draw calls). Two concrete implementations exist: GLDevice (OpenGL 4.6) and VKDevice (Vulkan). Render passes, materials, and the ECS are written against the abstract interface and work identically on both backends.
- **Deferred Rendering Pipeline:** A multi-pass renderer that decouples geometry from shading. A Geometry Pass fills a 4-attachment G-Buffer (world position, encoded normal, albedo/roughness, metallic/AO) and a Lighting Pass reads those textures to evaluate lighting on a full-screen quad. A Forward Pass handles translucent geometry and the skybox.
- **Disney Principled BSDF:** A physically-based shading model faithful to the Disney 2012 paper. Implemented in GLSL with separate lobes for diffuse (with retro-reflection), anisotropic specular (GTR2/GGX), clearcoat (GTR1), and sheen. Used in the deferred lighting pass and as a forward material.
- **Shadow Mapping with PCF:** Directional shadow maps generated with an orthographic light-space matrix. Soft shadows use a 3x3 Percentage Closer Filtering kernel.
- **Light Probes / Spherical Harmonics:** Diffuse indirect illumination approximated with L2 Spherical Harmonics (9 coefficients). Probes are baked at world positions and blended per-object by distance. Irradiance is evaluated in the shader using Ramamoorthi & Hanrahan projection constants.
- **Terrain System:** Procedural terrain with fractional Brownian Motion noise. Texture splatting by slope and height, high-frequency detail map with view-distance fade, real-time brush editing, and L-System vegetation instancing (5,000 GPU instances per draw call).

- **Frustum Culling:** Per-object frustum culling eliminates draw calls for geometry outside the camera frustum before submission to the render queue.
- **River Generation:** A* pathfinding on the heightfield finds a downhill river course. The channel is carved into the terrain with smoothstep blending; a ribbon water mesh is generated along the path.
- **Level of Detail (LOD):** Up to 4 mesh resolutions per entity. The correct LOD level is selected at render time based on camera distance, reducing GPU vertex load for distant geometry.
- **Lua Scripting System:** Per-entity Lua scripts (via sol2) with init/update/destroy lifecycle hooks. The engine API (ECS, Input, TransformComponent) is exposed to Lua.
- **GPU Instancing and Batching:** The render queue separates instanced commands from single-draw commands. Vegetation and other repeated geometry share one draw call per mesh type through an instance matrix SSBO.
- **Asynchronous Mesh Streaming:** loadOBJAsync() offloads OBJ parsing to the Job System. GPU upload is forwarded to the main thread via Dispatcher::RunOnMain(). The caller polls LoadState or provides a MeshReadyCallback.
- **Archetype-based ECS:** Entity-Component-System with archetype-level contiguous component storage. Adding or removing a component migrates the entity to a matching archetype using swap-and-pop.
- **Work-stealing Job System:** Each worker thread owns a lock-free bounded deque. Jobs wrapped in std::packaged_task are dispatched to a random worker and stealable by idle workers for load balancing.
- **Modular Shader System:** GLSL modules carry a MODULE_INFO header (name, stage, priority, dependencies). The compiler resolves the dependency graph and concatenates modules in priority order, allowing Disney BSDF to be declared once and reused by forward and deferred passes.
- **OpenGL 4.6 Direct State Access:** Uniform uploads use glProgramUniform* and VAO setup uses glVertexArrayVertexBuffer / glVertexArrayAttribFormat / glVertexArrayAttribBinding, removing all bind/unbind pairs.

2. Implemented Rendering Techniques

This section documents each rendering technique in tutorial style: what it is, why it is used, how it works mathematically, and how MAM Engine implements it — pointing directly to the relevant source files and GLSL code.

2.1 Deferred Rendering

Deferred rendering separates the scene into two independent passes so that the lighting cost scales with the number of screen pixels, not with the number of geometry-light pairs. This is the core rendering architecture of MAM Engine.

How it works

In the Geometry Pass every mesh is rasterised once and writes material data into a set of render targets called the G-Buffer. No lighting is computed at this stage. In the Lighting Pass a full-screen quad is drawn and, for each pixel, the G-Buffer textures are sampled to reconstruct the surface properties that the BSDF needs.

G-Buffer layout

Four colour attachments are allocated (Format::RGBA8):

- **Attachment 0 — gPosition:** World-space fragment position (xyz)
- **Attachment 1 — gNormal:** Encoded world-space normal (xyz * 0.5 + 0.5)
- **Attachment 2 — gAlbedoRoughness:** Albedo colour (rgb) and roughness (a)
- **Attachment 3 — gMetallicAO:** Metallic (r) and ambient occlusion (g)

Geometry Pass — gbuffer_frag.glsl

The fragment shader writes to all four targets. Normal mapping (see Section 2.3) is applied before encoding the normal into [0,1] range:

```
// Outputs: four bound render targets
layout(location = 0) out vec4 gPosition;
layout(location = 1) out vec4 gNormal;
layout(location = 2) out vec4 gAlbedoRoughness;
layout(location = 3) out vec4 gMetallicAO;

// After TBN normal mapping (see Section 2.3):
gPosition        = vec4(v_fragPos, 1.0);
```

```
 gNormal          = vec4(worldNormal * 0.5 + 0.5, 1.0);   // encode
[-1,1] -> [0,1]
 gAlbedoRoughness   = vec4(albedo, roughness);
 gMetallicAO      = vec4(metallic, 1.0, 0.0, 1.0);
```

Lighting Pass — deferred_light_frag.glsl

The lighting shader reads the G-Buffer and decodes each value. The normal decode is the inverse of the encoding: N = normalize(N * 2.0 - 1.0). Pixels with no geometry (normal length < 0.001) are discarded early, saving the full BSDF cost for background pixels.

```
vec3 fragPos   = texture(gPosition, v_texCoord).rgb;
 vec3 N           = texture(gNormal,   v_texCoord).rgb;
 if (length(N) < 0.001) discard;             // sky / empty pixel
 N = normalize(N * 2.0 - 1.0);                 // decode back to world
space

 vec3  albedo     = texture(gAlbedoRoughness, v_texCoord).rgb;
 float roughness = texture(gAlbedoRoughness, v_texCoord).a;
 float metallic  = texture(gMetallicAO,        v_texCoord).r;

 // Evaluate Disney BSDF for every light in the SSBO
 vec3 V  = normalize(u_cameraPos - fragPos);
 vec3 Lo = vec3(0.0);
 for (int i = 0; i < lightCount; i++)
    Lo += evaluateLight(lights[i], fragPos, N, V, albedo, roughness,
metallic);

 // Reinhard tonemapping + gamma correction
 vec3 color = Lo / (Lo + vec3(1.0));
 color = pow(color, vec3(1.0 / 2.2));
```

C++ implementation

The pipeline is managed by three pass objects: GeometryPass, LightingPass, and ForwardPass, all owned by the Renderer. The G-Buffer framebuffer is created with a 4-colour spec and a depth attachment; the lighting output framebuffer has a single colour target and no depth. Key files: render/api/geometry_pass.cpp, render/api/lighting_pass.cpp.

2.2 Disney Principled BSDF

Physically-based shading models describe how surfaces scatter light based on measurable physical quantities. MAM Engine implements the Disney Principled BSDF (Burley, 2012 / 2015) which unifies artists-friendly parameters (metallic, roughness, specularTint, sheen, clearcoat) with physically-grounded microfacet theory.

Microfacet theory overview

The specular BRDF is built from three factors: D (microfacet distribution), G (geometric attenuation / shadowing-masking), and F (Fresnel reflectance). The formula is:

```
f_specular = (F * D * G) / (4 * NdotL * NdotV)
```

D determines how microfacets are oriented. MAM uses Generalized Trowbridge-Reitz (GTR) with γ = 2 (GGX) for the primary specular lobe and γ = 1 for the clearcoat lobe. G uses the Smith GGX height-correlated shadowing-masking term.

Distribution functions — disney_bsdf.module.glsl

```
// GTR2 (GGX) — primary specular lobe (gamma=2)
float GTR2(float NdotH, float a) {
   float a2 = a * a;
   float t  = 1.0 + (a2 - 1.0) * NdotH * NdotH;
   return a2 / (PI * t * t);
}

// GTR1 — clearcoat lobe (gamma=1, fixed roughness ~0.1)
float GTR1(float NdotH, float a) {
   if (a >= 1.0) return INV_PI;
   float a2 = a * a;
   float t  = 1.0 + (a2 - 1.0) * NdotH * NdotH;
   return (a2 - 1.0) / (PI * log(a2) * t);
}

// Smith GGX geometric term
float smithG_GGX(float NdotV, float alphaG) {
   float a = alphaG * alphaG;
   float b = NdotV  * NdotV;
   return 1.0 / (NdotV + sqrt(a + b - a * b));
}
```

Anisotropic specular lobe

When the anisotropic parameter is non-zero, the roughness is split into two principal directions (ax, ay) using the surface tangent T and bitangent B. The GTR2Aniso variant and smithG_GGXAniso handle the per-direction sampling:

```
float aspect = sqrt(1.0 - 0.9 * anisotropic);
float ax = max(0.001, sqr(roughness) / aspect);
float ay = max(0.001, sqr(roughness) * aspect);

float D = GTR2Aniso(NdotH, HdotX, HdotY, ax, ay);
```

```
float G = smithG_GGXAniso(NdotL, LdotX, LdotY, ax, ay)
          * smithG_GGXAniso(NdotV, VdotX, VdotY, ax, ay);
```

Diffuse lobe with retro-reflection

Standard Lambertian diffuse (INV_PI) is augmented with a retro-reflection term that brightens grazing-angle surfaces at high roughness — a characteristic of rough cloth and diffuse plastics visible in the original Disney measurements:

```
vec3 evalDisneyDiffuse(vec3 baseColor, float roughness,
                       float NdotL, float NdotV, float LdotH) {
   float FL = schlickWeight(NdotL);
   float FV = schlickWeight(NdotV);
   // Retro-reflection: brightens at grazing angles when roughness is
high
   float RR       = 2.0 * roughness * LdotH * LdotH;
   float FRetro = RR * (FL + FV + FL * FV * (RR - 1.0));
   float Fd = (1.0 - 0.5 * FL) * (1.0 - 0.5 * FV);
   return baseColor * INV_PI * (Fd + FRetro);
}
```

Clearcoat and Sheen lobes

The clearcoat lobe simulates a thin dielectric clear coat layer (car paint, wood varnish) using GTR1 with a near-zero roughness and a fixed F0 of 0.04. The sheen lobe adds edge brightness for fabric and cloth materials:

```
// Clearcoat: GTR1 distribution, fixed rough ~0.1, F0 = 0.04
float evalDisneyClearcoat(float clearcoatGloss,
   float NdotL, float NdotV, float NdotH, float LdotH) {
   float gloss = mix(0.1, 0.001, clearcoatGloss);
   float D = GTR1(NdotH, gloss);
   float F = mix(0.04, 1.0, schlickWeight(LdotH));
   float G = smithG_GGX(NdotL, 0.25) * smithG_GGX(NdotV, 0.25);
   return (D * F * G) / max(4.0 * NdotL * NdotV, 0.001);
}

// Sheen: Schlick-weight edge brightening, tinted toward the base
colour
vec3 evalDisneySheen(vec3 baseColor, float sheen, float sheenTint,
float LdotH) {
   float luminance  = dot(baseColor, vec3(0.3, 0.6, 0.1));
   vec3 tint    = luminance > 0.0 ? baseColor / luminance :
vec3(1.0);
   vec3 sheenColor = mix(vec3(1.0), tint, sheenTint);
   return sheen * sheenColor * schlickWeight(LdotH);
}
```

Full BSDF combination

The evalDisney() function combines all lobes. The metallic parameter drives a linear interpolation between dielectric and conductor behaviour: diffuse and sheen are zeroed out as metallic → 1, while F0 transitions from the dielectric value (0.04 or specular tinted) to the base colour itself.

```
vec3 diffuse      = evalDisneyDiffuse(...)  * (1.0 - metallic);
 vec3 sheenContrib = evalDisneySheen(...)    * (1.0 - metallic);
 vec3 spec         = evalDisneySpecular(...);
 float coat        = 0.25 * clearcoat * evalDisneyClearcoat(...);

 return (diffuse + sheenContrib + spec) * NdotL + vec3(coat * NdotL);
```

2.3 Normal Mapping

A normal map stores per-texel surface normals in tangent space (encoded as RGB). At runtime, a per-vertex TBN matrix transforms those normals into world space so that the lighting model operates on geometry that appears to have far more detail than the actual polygon mesh.

TBN construction — gbuffer_frag.glsl

The vertex shader interpolates the tangent vector (plus a handedness sign in the w component). In the fragment shader the Gram-Schmidt process re-orthogonalises T against N to correct for interpolation drift:

```
vec3 N = normalize(v_normal);

 // Re-orthogonalise tangent using Gram-Schmidt
 vec3 T = normalize(vec3(v_tangent));
 T = normalize(T - dot(T, N) * N);

 // Bitangent — sign stored in v_tangent.w handles mirrored UVs
 vec3 B = cross(N, T) * v_tangent.w;

 mat3 TBN = mat3(T, B, N);
```

Applying the normal map

The sampled texel is in [0,1]; it must be unpacked to [-1,1] before transforming it into world space with the TBN matrix. The result is written to gNormal for the lighting pass to use:

```
if (u_useNormalMap != 0) {
    vec3 normalTS = texture(u_normalMap, v_texCoord).rgb;
    normalTS = normalTS * 2.0 - 1.0;     // [0,1] -> [-1,1]
    worldNormal = normalize(TBN * normalTS); // tangent -> world space
```

```
}
gNormal = vec4(worldNormal * 0.5 + 0.5, 1.0); // encode for G-Buffer
```

2.4 Shadow Mapping with PCF

Shadow mapping renders the scene from the light's point of view to produce a depth texture, then compares each fragment's light-space depth against the stored value to determine shadowing. Percentage Closer Filtering (PCF) samples multiple neighbours and averages the comparison results to produce soft shadow edges.

Shadow pass

The shadow vertex shader transforms each vertex into light space using the light-space matrix (orthographic for directional lights). No fragment shader output is needed — only the depth buffer is populated:

```
// shadow_vert.glsl
 gl_Position = u_lightSpace * u_model * vec4(a_position, 1.0);
```

PCF receiver — shadow_receiver.module.glsl

In the lighting pass, getShadowFactor() projects the fragment position into NDC light space, applies a fixed bias of 0.02 to avoid self-shadowing artefacts, then samples a 3×3 grid of texels. texelSize is computed from the actual texture dimensions so the kernel size adapts automatically to any shadow map resolution:

```
float getShadowFactor(vec3 fragPos, vec3 normal, vec3 lightDir) {
    vec4 fragPosLightSpace = u_lightSpace * vec4(fragPos, 1.0);
    vec3 projCoords = fragPosLightSpace.xyz / fragPosLightSpace.w;
    projCoords = projCoords * 0.5 + 0.5;   // NDC -> [0,1]

    if (projCoords.z > 1.0) return 1.0; // outside frustum = lit

    float bias = 0.02;                  //    fixed    bias    against
self-shadow

    // PCF: 3x3 kernel — 9 samples, average the results
    float shadow   = 0.0;
    vec2  texelSize = 1.0 / textureSize(u_shadowMap, 0);
    for (int x = -1; x <= 1; ++x)
    for (int y = -1; y <= 1; ++y) {
           float pcfDepth = texture(u_shadowMap,
           projCoords.xy + vec2(x, y) * texelSize).r;
           shadow += (projCoords.z - bias) > pcfDepth ? 0.0 : 1.0;
    }
```

```
    return shadow / 9.0;    // 1.0 = fully lit, 0.0 = fully shadowed
}
```

2.5 Terrain Splatting with Detail Map

Terrain texturing blends multiple ground textures based on the slope and height of each surface point — a technique called texture splatting. A separate high-frequency detail map adds small-scale surface variation that fades out as the viewer moves away.

Slope and height blending — terrain_splatting.module.glsl

Slope is derived from the Y component of the world normal: a flat surface has v_worldNormal.y ≈ 1 (slope ≈ 0); a vertical cliff has v_worldNormal.y ≈ 0 (slope ≈ 1). Height blending transitions a third texture at higher elevations (snow, rock cap):

```
// slope: 0 = flat, 1 = vertical
 float slope      = 1.0 - v_worldNormal.y;
 float slopeBlend  = smoothstep(u_slopeMin, u_slopeMax, slope);
 float heightBlend = smoothstep(u_heightLow, u_heightHigh, v_worldY);

  vec3 color0 = sampleTerrain(u_texture0, v_worldXZ, u_texScale); //
base (grass)
  vec3 color1 = sampleTerrain(u_texture1, v_worldXZ, u_texScale); //
slope (rock)
  vec3 color2 = sampleTerrain(u_texture2, v_worldXZ, u_texScale); //
high (snow)

 vec3 color = mix(color0, color1, slopeBlend);
    color = mix(color, color2, heightBlend);
```

Detail map with view-distance fade

A detail texture at a higher UV scale adds micro surface detail (pores, gravel grains). It is applied as a signed offset so it never permanently brightens or darkens the base. The weight fades smoothly to zero beyond u_detailFadeEnd metres, avoiding visible texture pop at a distance:

```
vec3 detail      =       texture(u_texture3,       v_worldXZ       *
u_detailScale).rgb;
 float viewDist  = 1.0 / gl_FragCoord.w;   // approximate linear depth
    float    detailFade    =    1.0    -    smoothstep(u_detailFadeStart,
u_detailFadeEnd, viewDist);
 float weight     = u_detailStrength * detailFade;

 // Signed offset: detail 0.5 -> no change; < 0.5 -> darken; > 0.5 ->
brighten
```

```
color += (detail - vec3(0.5)) * 2.0 * weight;
color  = clamp(color, 0.0, 1.0);
```

Procedural generation and brush editing

The terrain heightfield is generated from fractal Brownian Motion (fBm) Perlin noise (TerrainNoiseParams: octaves, persistence, redistribution, frequency). The editor provides real-time brush tools (Raise / Lower / Smooth) controlled via an ImGui panel. Vegetation is scattered via an L-System (VegetationInstancer) with 5 000 GPU instances uploaded once to an instance SSBO.

2.6 Light Probes and Spherical Harmonics

Direct lighting gives each surface the colour of the lights that hit it, but in the real world surfaces also receive diffuse indirect light bouncing off surrounding geometry. MAM Engine approximates this with Light Probes — spheres placed in the world that store a compressed representation of the incoming radiance as L2 Spherical Harmonics coefficients.

Spherical Harmonics background

Spherical Harmonics are an orthonormal basis over the sphere analogous to a Fourier series. The L2 (order 2) band has 9 basis functions per colour channel. Projecting a lighting environment onto these 9 coefficients compresses what would be a full cube-map into 27 floats (3 channels × 9). Reconstruction is a dot product between the coefficients and the analytical integrals of the SH basis functions weighted by a cosine lobe (the Lambertian transfer function), as derived by Ramamoorthi & Hanrahan 2001.

Irradiance evaluation — pbr_lighting.module.glsl

The 9 pre-scaled SH coefficients are uploaded per-object via uniform u_sh[9]. The evaluation constants (c0..c4) embed both the SH basis normalisation and the Lambertian integral weights:

```
vec3 evaluateProbeIrradiance(vec3 n) {
    float x = n.x, y = n.y, z = n.z;

    // Ramamoorthi & Hanrahan transfer coefficients
    const float c0 = 0.282095 * 3.141593;   // l=0  (DC term)
    const float c1 = 0.488603 * 2.094395;   // l=1  (linear)
    const float c2 = 1.092548 * 0.785398;   // l=2  cross terms
    const float c3 = 0.315392 * 0.785398;   // l=2  Y6
    const float c4 = 0.546274 * 0.785398;   // l=2  Y8

    vec3 irr = vec3(0.0);
```

```
    irr += u_sh[0] * c0;
    irr += u_sh[1] * (c1 * y);
     irr += u_sh[2] * (c1 * z);
    irr += u_sh[3] * (c1 * x);
    irr += u_sh[4] * (c2 * x * y);
    irr += u_sh[5] * (c2 * y * z);
    irr += u_sh[6] * (c3 * (3.0 * z * z - 1.0));
    irr += u_sh[7] * (c2 * x * z);
    irr += u_sh[8] * (c4 * (x * x - y * y));
    return max(irr, vec3(0.0));
}
```

Probe blending

The CPU side (forward_pass.cpp, computeBlendedProbeSH) iterates all probes, computes an inverse-distance weight clamped to the probe radius, and accumulates a weighted sum of coefficients. If no probe covers the object the nearest probe is used as a fallback.

```
// Per-probe contribution: weight = (1 - dist/radius)^2
 float w = 1.0f - glm::clamp(dist / radius, 0.0f, 1.0f);
 w = w * w;
 for (int c = 0; c < kSHCoeffCount; ++c)
    outSH[c] += glm::vec3(probe.sh[c]) * w;
 // ... then normalise by totalWeight
```

2.7 Reinhard Tonemapping and Gamma Correction

HDR rendering accumulates light in linear floating-point buffers where values can exceed 1.0. Before the image is displayed on an 8-bit monitor two steps are needed: tonemapping (compress HDR → [0,1]) and gamma correction (convert linear energy values to the non-linear gamma-encoded signal that monitors expect).

Implementation — deferred_light_frag.glsl

MAM Engine applies the Reinhard operator followed by a 2.2 gamma curve. These two lines run at the end of the deferred lighting pass, after the full BSDF accumulation, so tonemapping is always the last operation on the linear signal:

```
// Reinhard tonemapping: maps [0, +inf) to [0, 1)
 // Each channel is compressed independently; bright areas desaturate
slightly
 color = color / (color + vec3(1.0));
```

```
// Gamma correction: linear -> sRGB  (monitor gamma ≈ 2.2)
color = pow(color, vec3(1.0 / 2.2));
```

Reinhard tonemapping has the useful property that it never clips: no matter how bright a pixel is, color / (color + 1) stays below 1.0. The gamma correction step compensates for the monitor's transfer curve, ensuring that a linear 50% grey appears as perceived 50% grey rather than as ~22% luminance.

2.8 Vegetation Instancing

Drawing thousands of trees or grass blades with individual draw calls is prohibitively expensive. GPU instancing allows a single draw call to render N copies of the same mesh by reading per-instance data from an attribute buffer or SSBO.

Instance matrix in vertex shader, vegetation_transform_vert.module.glsl

Each instance carries a full 4×4 world matrix (a_instanceMatrix), uploaded once as an instance-rate attribute buffer. The vertex shader reads it to transform the position and recomputes the normal matrix to avoid non-uniform scale artefacts on normals:

```
// Each instance provides its own model matrix via instance attribute
vec4 worldPos4  = a_instanceMatrix * vec4(a_position, 1.0);
gl_Position     = u_projection * u_view * worldPos4;
v_worldPos      = worldPos4.xyz;
v_texCoord      = a_texCoord;

// Normal matrix: transpose of the inverse of the upper-left 3x3
// Handles non-uniform scale without distorting normals
mat3 normalMat  = transpose(inverse(mat3(a_instanceMatrix)));
v_normal        = normalize(normalMat * a_normal);
```

L-System tree generation

Tree positions and orientations are generated with an L-System (axiom + production rules + angle + iterations). The VegetationInstancer bakes up to 5 000 instance matrices into a vertex buffer and issues a single glDrawArraysInstanced call through gd_.drawInstanced(). Key files: workspaces/08_terrain/08_terrain.cpp, render/api/forward_pass.cpp (drawOpaqueInstanced).

2.9 Frustum Culling

Frustum culling eliminates draw calls for geometry that lies entirely outside the camera's view frustum before those calls ever reach the GPU. This

saves vertex shader work, rasterisation bandwidth, and driver overhead — especially important for large open-world scenes.

Extracting frustum planes — engine_systems.cpp

The camera's combined view-projection matrix (VP) encodes all six frustum planes as linear combinations of its rows. The Gribb/Hartmann method extracts each plane directly from VP without needing the frustum corner vertices. Each resulting plane is normalised so that the signed distance formula dot(normal, point) + d gives the true distance from the plane:

```
static std::array<FrustumPlane, 6>
 extractFrustumPlanes(const glm::mat4& vp)
 {
    // Extract rows from column-major matrix
    glm::vec4 row0 = { vp[0][0], vp[1][0], vp[2][0], vp[3][0] };
    glm::vec4 row1 = { vp[0][1], vp[1][1], vp[2][1], vp[3][1] };
    glm::vec4 row2 = { vp[0][2], vp[1][2], vp[2][2], vp[3][2] };
    glm::vec4 row3 = { vp[0][3], vp[1][3], vp[2][3], vp[3][3] };

    // Gribb/Hartmann: each frustum plane is a row combination
    // Plane is normalised so |normal| = 1
    return {
            makePlane(row3 + row0), // Left
            makePlane(row3 - row0), // Right
            makePlane(row3 + row1), // Bottom
            makePlane(row3 - row1), // Top
            makePlane(row3 + row2), // Near
            makePlane(row3 - row2), // Far
    };
 }
```

Visibility tests

MAM Engine supports two bounding volume types. The sphere test is O(6 dot products). The AABB test uses the positive vertex trick — for each plane, the corner of the box that is furthest in the plane's normal direction is the vertex most likely to be inside. If even that vertex is outside the plane, the entire AABB is outside:

```
// Sphere: outside if center is farther than -radius from any plane
 static bool isSphereVisible(const glm::vec3& center, float radius,
    const std::array<FrustumPlane, 6>& planes)
 {
    for (const auto& p : planes)
            if (glm::dot(p.normal, center) + p.d < -radius) return
false;
    return true;
 }
```

```
// AABB: positive-vertex test — worst-case corner per plane
static bool isAABBVisible(const glm::vec3& min, const glm::vec3& max,
   const std::array<FrustumPlane, 6>& planes)
{
   for (const auto& p : planes) {
          glm::vec3 pv = {             //  pick  corner  most  aligned
with plane normal
          p.normal.x >= 0.f ? max.x : min.x,
          p.normal.y >= 0.f ? max.y : min.y,
          p.normal.z >= 0.f ? max.z : min.z,
          };
          if  (glm::dot(p.normal,  pv)  +  p.d  <  0.f)  return  false;  //
fully outside
   }
   return true;
}
```

Integration in the render system

updateRenderSystem() computes the frustum once per frame from the current VP matrix, then tests every entity with a RenderComponent before submitting a draw command. If RenderComponent carries a valid bounding sphere radius the sphere test runs first (cheaper); if it also carries an AABB the AABB test is applied as a second pass. Only entities that pass both tests are submitted to the RenderQueue.

```
auto frustumPlanes = extractFrustumPlanes(projection * view);
// Per entity:
if (rc.boundingSphereRadius > 0.f)
   if (!isSphereVisible(rc.boundingSphereCenter,
                              rc.boundingSphereRadius, frustumPlanes))
continue;
if (rc.aabbMin != rc.aabbMax)
   if    (!isAABBVisible(rc.aabbMin,    rc.aabbMax,    frustumPlanes))
continue;
// ... submit draw command
```

2.10 Level of Detail (LOD)

Level of Detail reduces GPU vertex processing load for distant objects by substituting lower-polygon mesh variants when the object is far from the camera. The reduction in vertex count for distant objects has no perceptible visual impact since those objects cover only a few pixels on screen.

LODComponent data structure

Each entity that needs LOD carries a LODComponent with up to four mesh pointers ordered from highest to lowest detail, four distance thresholds, and the object's local-space centre for distance calculation:

```
struct LODComponent {
    static constexpr int MAX_LEVELS = 4;
    std::array<Mesh*, MAX_LEVELS>  meshes;      //        LOD0=highest,
LOD3=lowest
    std::array<float, MAX_LEVELS>  distances;  // switch distances from
camera
    glm::vec3                      center;       //   local-space   pivot
for dist calc
    int                            levels = MAX_LEVELS;
 };
```

LOD selection — engine_systems.cpp

At render time, updateRenderSystem() checks whether the entity has a LODComponent. If it does, the camera-to-centre distance is computed and the first level whose threshold exceeds that distance is selected. This replaces the default mesh in the draw command without any other changes to the render pipeline:

```
if (lods) {
    auto& lod  = lods->get(i);
    float dist = glm::distance(camera->position(), lod.center);

    // Start from lowest detail, walk toward highest
    selectedMesh = lod.meshes[lod.levels - 1]; // default: lowest
    for (int l = 0; l < lod.levels; ++l) {
           if (dist < lod.distances[l]) {
           selectedMesh = lod.meshes[l];       // use finest mesh that
fits
           break;
           }
    }
 }
```

Because LOD selection happens in the ECS system before the draw command is submitted, no changes are needed in the render passes themselves. The forward pass and deferred pass draw whichever Mesh pointer they receive.

2.11 River Generation

Rivers are procedurally generated in two steps: an A* search on the terrain heightfield finds a realistic downhill path from source to sink, then the heightfield is carved along that path and a water ribbon mesh is generated. The river is fully integrated into the terrain — its bed is physically below the surrounding ground and the banks blend smoothly into the landscape.

A* pathfinding on the heightfield — river.cpp

The heightfield grid is the A* cost space. Moving uphill incurs an extra penalty proportional to the height increase (uphillPenalty), so the path naturally follows valleys. Diagonal moves cost sqrt(2) to account for actual grid distance. The heuristic is the octile distance — the optimal grid heuristic for 8-connected movement:

```
// Octile distance: handles diagonal moves exactly
 inline float octile(glm::ivec2 a, glm::ivec2 b) {
    int dx = std::abs(a.x - b.x), dz = std::abs(a.y - b.y);
    return float(std::max(dx,dz) - std::min(dx,dz))
           + 1.41421356f * float(std::min(dx,dz));
 }

 // Step cost: base distance + uphill penalty
 float dh         = field[ni] - curH;          // height difference
 float stepCost = base;                           // 1.0 (cardinal) or
sqrt(2) (diagonal)
 if (dh > 0.0f) stepCost += params.uphillPenalty * dh;   // penalise
climbing

 // f = g + h*weight  (weight >= 1.0 trades optimality for speed)
 float fScore = tentativeG + params.heuristicWeight * octile({nx,nz},
sink);
```

For performance on large terrains, pathfinding runs on a downsampled coarse grid (controlled by pathStride). The resulting coarse path is then upsampled with Bresenham-style line interpolation to produce a dense pixel-accurate path.

Terrain carving

Once the path is known, the riverbed elevation is computed as a running minimum of (surface height - depth) along the path. This prevents the river from "climbing" back up. The carved channel uses a smoothstep blend so banks grade naturally:

```
// Running minimum ensures the bed never goes uphill
 float running = orig[path[0]] - carve.depth;
 for (size_t i = 0; i < path.size(); ++i) {
    running = std::min(running, orig[path[i]] - carve.depth);
    bed[i]  = running;
 }

 // Per path point: carve a circular channel with smoothstep falloff
 float radius = (carve.width * 0.5f) / cellSize;
 float k = dist / radius;
 float s = k*k*(3.0f - 2.0f*k);              // smoothstep blend:
0=center, 1=bank
 float t = bedH * (1.0f - s) + bank * s; // interpolate bed to bank
height
```

```
 field[id] = std::min(field[id], t);       // only lower, never raise
terrain
```

Water ribbon mesh

buildWaterMesh() generates a quad-strip ribbon mesh that follows the carved path. At each point, the local tangent is computed from neighbouring path points; the perpendicular gives the ribbon width. UVs scroll along the V axis for flow animation:

```
// Per path point: compute tangent from neighbours
 glm::vec2 dir   = glm::normalize(path[ib] - path[ia]);   // smooth
tangent
 glm::vec2 perp = { -dir.y, dir.x };                  // perpendicular
= ribbon edge
 float y          = bed[i] + wp.waterRaise;           // water surface
height

 // Left and right vertices of the ribbon
 vl.position = glm::vec3(cur - perp * halfW, y);
 vr.position = glm::vec3(cur + perp * halfW, y);
 vl.texCoord = glm::vec2(0.0f, float(i));  // V scrolls for animation
 vr.texCoord = glm::vec2(1.0f, float(i));
```

2.12 SSAO

What is it?

SSAO is a rendering technique that approximates ambient occlusion in screen space. It simulates how corners, crevices, and areas near other geometry receive less ambient light, adding depth and realism to the scene without the need for costly global calculations.

Implementation

The implementation is divided into two passes:
1. SSAO (raw) pass A kernel of hemispherical samples is generated in view space. For each fragment, KERNEL_SIZE samples are taken around the G-Buffer normal, and they are checked for occlusion by comparing their depth to the actual scene. The samples are distributed with quadratic acceleration to concentrate them near the fragment:

```
float t = static_cast<float>(i) / static_cast<float>(KERNEL_SIZE);
sample *= glm::mix(0.1f, 1.0f, t * t);
```

To avoid banding, the kernel is randomly rotated using a 4×4 noise texture that repeats across the screen (tiled with noiseScale = screenSize / 4.0).

2. Blur Pass The raw result of the SSAO contains noise due to stochastic sampling. A blur is applied to smooth it before using it in the lighting pass.

G-Buffer Inputs

- gPosition — position in view space (attachment 0)
- gNormal — normal in view space (attachment 1)
- u_noiseTexture — 4×4 noise texture to rotate the kernel

Profiling

GPU Timer Queries (GL_TIME_ELAPSED) are used to measure the cost of the pass in milliseconds, accessible via lastPassMs_.

3. Strategies and Solutions

This section describes the architectural decisions made during development, the problems encountered, and how they were resolved.

3.1 Backend-agnostic Render API and Vulkan Backend

A GraphicsDevice abstract interface defines all GPU operations as pure virtual methods. Two concrete backends implement it: GLDevice (OpenGL 4.6) and VKDevice (Vulkan). All render passes are written against the abstract interface and work on both backends.

VKDevice — Vulkan resource lifecycle

```
// Initialization order in VKDevice constructor
createInstance();        // VkInstance + validation layers
setupDebugCallback();    // VkDebugUtilsMessengerEXT
createSurface();         // VkSurfaceKHR from GLFW window
pickPhysicalDevice();    // select best VkPhysicalDevice
createLogicalDevice();   // VkDevice + queue handles
createSwapChain();       // VkSwapchainKHR + images
createImageViews();      // VkImageViews
createRenderPass();      // default VkRenderPass
createFramebuffers();    // per-image VkFramebuffers
createCommandPool();     // VkCommandPool
createCommandBuffers(); // per-frame VkCommandBuffers
createSyncObjects();     // semaphores + fences
```

VKPipeline — descriptor sets and uniform management

Vulkan requires all shader inputs declared in a pipeline layout via descriptor sets. VKPipeline uses set 0 for the UBO (MVP + Disney BSDF parameters matching the UniformBufferObject struct in std140 layout) and set 1 for per-material textures. Per-material descriptor sets are cached by texture pointer combination so draw calls sharing the same textures reuse one set. A dummy 1x1 white texture fills unbound sampler slots to satisfy Vulkan validation requirements.

Runtime backend detection in render passes

```
const bool isVulkan = dynamic_cast<VKDevice*>(&gd_) != nullptr;
if (isVulkan) {
   auto* vkSSBO = dynamic_cast<VKStorageBuffer*>(lightBuffer_.get());
    vkSSBO->uploadData(ctx.lights.data(), sizeBytes, 0);
    vkSSBO->bindToPipeline(vkPipeline);
} else {
   lightBuffer_->bind();
```

```
    lightBuffer_->uploadData(ctx.lights.data(), sizeBytes, 0);
}
```

3.2 Archetype-based ECS

An Entity-Component-System architecture was chosen to cleanly separate data (components) from logic (systems) and to allow the engine to iterate large sets of entities efficiently. We went with an archetype model rather than a sparse-set model because component access patterns are predictable in a game engine — iterating all entities with a Transform + Mesh pair is the most common query, and archetypes guarantee those components are stored contiguously in memory.

Challenge: when a component is added or removed the entity must migrate to a different archetype. We solved this with a swap-and-pop strategy: the entity's components are copied into the new archetype's arrays, the vacated slot is filled by moving the last entity in the old archetype, and the entity-location table is updated atomically. Migration cost is O(k) where k is the number of component types, which is acceptable for the infrequent add/remove calls typical of game code.

3.3 Work-stealing Job System

CPU-heavy tasks (mesh loading, terrain generation, probe baking) block the main thread if run synchronously. We implemented a multi-threaded job system where each worker thread owns a bounded lock-free deque. Jobs are dispatched to a randomly selected worker and may be stolen from the top of any queue by an idle worker, providing automatic load balancing without central coordination.

submit_callable() wraps an arbitrary callable in a std::packaged_task, stores the shared_future in a JobHandle, and pushes the job to the worker's deque. If the target deque is full the job is tried on other workers, and as a last resort executed inline on the calling thread to avoid deadlock.

3.4 Asynchronous Mesh Streaming

OBJ files can be multi-megabyte; parsing them synchronously causes visible frame drops during level loads. loadOBJAsync() submits the file read and parse job to the Job System. The critical design constraint is that OpenGL GPU uploads must happen on the main thread (the thread that owns the GL context). We solved this with the Dispatcher pattern:

```
// Worker thread: parse the OBJ (no GL calls)
auto jobHandle = jobSystem.submit_callable([path, onReady](){
   auto raw = parseOBJ(path);
   // Forward GPU upload to the main thread
   Dispatcher::RunOnMain([raw, onReady](){
          auto mesh = uploadToGPU(raw);
          if (onReady) onReady(mesh);
   });
});

// Main thread (once per frame)
DrainMainQueue(); // executes all queued GPU uploads
```

The LoadState enum (Pending / Ready / Failed) allows the application to poll or to react via the MeshReadyCallback without blocking. This pattern is particularly visible in the dispatcher.hpp and mesh.hpp headers.

3.5 Lua Scripting System

Game logic is separated from engine code by exposing a Lua API (via sol2). Each entity can carry a ScriptComponent pointing to a .lua file. The ScriptSystem loads the script on first access and calls init(entity), update(entity, dt), and destroy(entity) at the appropriate lifecycle points. The engine exposes ECS operations (createEntity, addTransform, getTransform), Input (isKeyPressed), and glm::vec3 to Lua. This allows game-specific behaviour to be written in Lua without recompiling the engine.

3.6 Modular Shader System

A naive approach concatenates all shader code into monolithic files, leading to duplication when the same technique (e.g. Disney BSDF) is used in multiple passes. We designed a MODULE_INFO header convention: each .glsl file declares its name, stage, priority, and dependencies. At startup the shader compiler resolves the dependency graph, sorts modules by priority, and concatenates them into the final source string.

This allowed the disney_bsdf module (priority 5) to be declared once and referenced by pbr_lighting (priority 10) which declares it as a dependency. The forward pass and deferred lighting pass both use the same BSDF module without any copy-paste.

3.7 OpenGL 4.6 Direct State Access

Legacy OpenGL requires objects to be bound before modification, which creates implicit global state and makes code order-dependent. OpenGL 4.5+ Direct State Access (DSA) removes this constraint by naming the object in every call.

All uniform uploads in gl_pipeline.cpp use glProgramUniform* so the pipeline does not need to be bound when setting parameters. The VAO setup in gl_vertex_array.cpp uses glVertexArrayVertexBuffer, glVertexArrayAttribFormat, glVertexArrayAttribBinding, and glVertexArrayElementBuffer, no glBindVertexArray / glBindBuffer pair needed during attribute setup.

```
// gl_vertex_array.cpp — full DSA setup, no bind/unbind required
 glEnableVertexArrayAttrib  (id_, index);
 glVertexArrayVertexBuffer  (id_, index, bufferId, 0, stride);
 glVertexArrayAttribFormat   (id_, index, count, GL_FLOAT, normalized,
relOffset);
 glVertexArrayAttribBinding (id_, index, index);

 // gl_pipeline.cpp — uniform set without binding the pipeline
 glProgramUniform3fv(id_, loc, 1, static_cast<const GLfloat*>(value));
```

3.8 Runtime Shader Compilation Warning (LNK4006)

LNK4006 — __NULL_IMPORT_DESCRIPTOR duplicate symbol
During linking, the following warning is emitted:

shaderc_shared.lib(shaderc_shared.dll) : warning LNK4006: __NULL_IMPORT_DESCRIPTOR already defined in vulkan-1.lib(vulkan-1.dll);
second definition ignored

This is a benign linker warning that arises because both vulkan-1.lib and shaderc_shared.lib are import libraries built from DLLs. Each import library contains a sentinel symbol (__NULL_IMPORT_DESCRIPTOR) that marks the end of its import table. When the linker merges both libraries into the same binary it encounters the symbol twice and emits the warning, but silently discards the duplicate — the resulting executable is correct.

The warning cannot be suppressed without removing one of the two dependencies. shaderc is required for runtime GLSL → SPIR-V compilation in VKShader::compile(), which allows shaders to be authored and hot-reloaded as plain .glsl source files without a manual offline compilation step. Replacing it with pre-compiled SPIR-V blobs would eliminate the warning but would remove the ability to compile shaders at runtime, breaking the modular shader system described in Section 3.4. The warning is therefore accepted as a known, harmless artefact of the build configuration.

4. Task Distribution

The project was developed over the course of the semester as a three-person team. The following table summarises the primary areas of responsibility for each member. All members participated in code reviews, debugging sessions, and final integration.

Member	Primary Responsibilities	Key Files / Modules
Alicia de Cubas	Vulkan backend integration: VKDevice lifecycle, swapchain creation and recreation, VKPipeline descriptor sets and uniform management, VKBuffer/VKTexture/VKFramebuffer with memory allocation and resource management, VKShader with SPIR-V compilation. Light probe system: ProbeBaker with SH2 projection and ProbeInterpolator with inverse-distance	vk_pipeline.hpp/cpp, vk_buffer.hpp/cpp, vk_texture.hpp/cpp, vk_framebuffer.hpp/cpp, vk_shader.hpp/cpp, vk_vertex_array.hpp/cpp, vk_context.hpp/cpp, vk_window.hpp/cpp, light_probe.hpp, probe_systems.hpp, probe_baker.hpp/cpp, probe_interpolator.hpp/cpp
Matias Pedraza	Terrain system: procedural generation (fBm noise), texture splatting, brush editor (Raise/Lower/Smooth), river generation. Vegetation instancing with L-System. Frustum culling. GPU instancing and draw-call batching. Job System design and implementation. Asynchronous mesh streaming (Dispatcher / DrainMainQueue / LoadState). Scripting system.	08_terrain.cpp, terrain_splatting.module.glsl, vegetation_transform_vert.module.glsl, job_system.hpp/cpp, dispatcher.hpp, mesh.hpp/cpp (loadOBJAsync)
Mark Syniato	Archetype-based ECS (World, Archetype, ArchetypeRegistry, component migration). . OpenGL base implementation improvements: DSA refactor (gl_pipeline.cpp, gl_vertex_array.cpp), std::optional error handling, RAII cleanup, const-correctness, make_unique migration. Deferred rendering pipeline (G-Buffer, Geometry Pass, Lighting Pass). Normal mapping. Reinhard tonemapping	world.hpp/cpp, archetype.hpp/cpp, pbr_lighting.module.glsl, gl_pipeline.cpp, gl_vertex_array.cpp, gl_buffer.hpp/cpp gbuffer_frag.glsl, deferred_light_frag.glsl, lighting_pass.cpp, geometry_pass.cpp, disney_bsdf.module.glsl,

Shared Responsibilities	OpenGL base layer implementation	gl_device.hpp/cpp, gl_frame_buffer.hpp/cpp, gl_pipeline.hpp, gl_shader.hpp/cpp, gl_texture.hpp/cpp, gl_vertex_array.hpp, gl_window.hpp/cpp, gl_buffer.hpp/cpp, gl_context.hpp/cpp, gl_cubemap.hpp/cpp